

Cyber Security and Digital Forensics

Data Science for Cyber Security



Submitted by

Name: Ajaj Ahmed

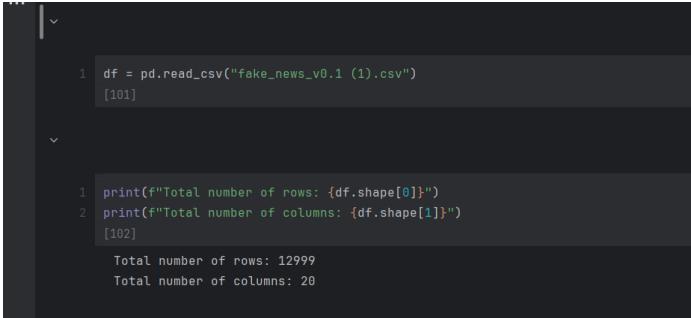
Data Science for Cyber Security

Kathmandu, Nepal

January 09, 2026

Task 1

	Task	Comments
--	------	----------

1	Load the dataset, find out and print the total number of rows and columns in the dataset.	Step1: - To perform this task some important libraries of python are imported that is pandas, numpy, matplotlib, sns, etc. - Loaded fake_news_v0.1 (1).csv using pandas' library. Step2: - After loading these files then uses .shape attribute to verify the dimensions of the datasets. - To print the total number of rows and columns I use the given technique and run the given python script in the screenshot. - df.shape(0) provides the rows count and df.shape(1) provides the columns count.  <pre> 1 df = pd.read_csv("fake_news_v0.1 (1).csv") [101] 1 print(f"Total number of rows: {df.shape[0]}") 2 print(f"Total number of columns: {df.shape[1]}") [102] Total number of rows: 12999 Total number of columns: 20 </pre> Analysis and Result: - after loading and printing the shape using {df.shape[0]} and {df.shape[1]} provides the outputs of the total rows and columns respectively - total number of rows: 12999 - total number of columns: 20 Commands: <pre> df = pd.read_csv("Fake_news_v0.1 (1).csv") print(f"total number of rows: {df.shape[0]}) print(f"total number of Columns: {df.shape[1]}) </pre>
---	--	--

2 Your dataset will have one or more columns containing dates, find out and print how many dates are null, incorrect or incomplete. Apply the dataset context and domain knowledge while investigating 'incorrect' dates.

Step2:

This part of analysis acts as the quality control check for the tempered and empty data. In this part I did two step of verification for checking null, incorrect and incomplete datasets.

- To check null values I have used `.isna().sum()` to count all the missing entries. After executing this for both crawled and published output shows that there are 0 null values for both published and crawled.
- Since there are not any null dates I proceed to identify the incorrect and invalid checks.
- To check incorrect and invalid I used `pd.to_datetime()` with `errors='correc'`, this script to turn script attempts to turn strings into actual datetime objects and turns into `nat`.
- After executing this code, I found that there is 36 invalid and incorrect dates. While crawled date are valid.

```
To Detect Null, Incorrect, Incomplete Dates

df
[100]

# rows = 1000 rows x 10 cols
# columns = 10 columns

# Published Date
# Crawled Date
```

```
Printing Invalid and Incorrect Dates

published_invalid = pd.to_datetime(df['published'], errors='coerce', utc=True).isna().sum()
crawled_invalid = pd.to_datetime(df['crawled'], errors='coerce', utc=True).isna().sum()

print("Invalid and incorrect Published Date:", published_invalid)
print("Invalid and incorrect Crawled Date :", crawled_invalid)

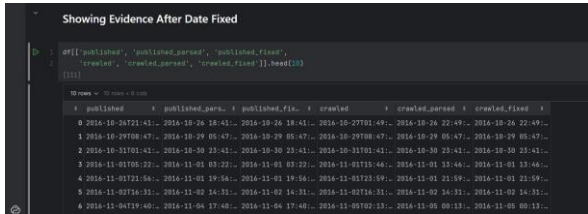
[100]

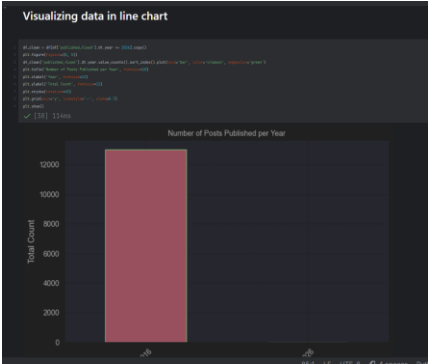
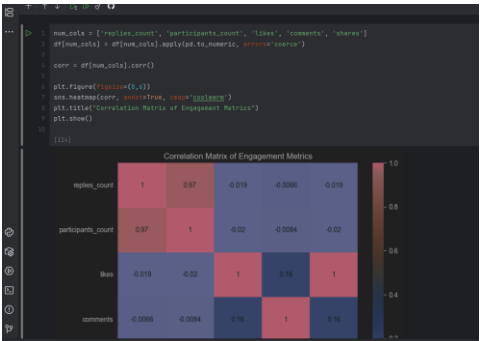
Invalid and incorrect Published Date: 36
Invalid and incorrect Crawled Date : 0
```

Fixing Incorec and Invalid Dates

Parsing Dates (Invalid -> Nat

```
df['published_parsed'] = pd.to_datetime(df['published'], errors='coerce', utc=True)
[50] 79ms
```

3	Fix the incorrect dates. Provide (print, visualise) evidence of fixed dates.	Step3: After identifying the null, incorrect and invalid date I proceed to fix these errors. • <code>Pd.to_datetime(df['published'], errors= 'correc', utc=true)</code>. This command tries to turn all the available strings into strings to date and into NaT. • Instead of overwriting the original data created a new column 'published_parsed'.  Dates are printed after fixing, by running the given script in the image only original date given in dataset and new created column of published_fixed, and crawled_fixed is printed which shows evidence of fixing the dates after findings.  After fixing the complete dates it is visualized in the line graph which shows that the publication of the article becomes very low after year 2016.

		It shows that there high usage of publication in 2016. 
4	Prepare and visualise a correlation matrix between the following numerical columns, - 'replies_count' - 'participants_count' - 'likes' - 'comments' - 'shares'	Correlation matrix shows the most usage of the dataset, it clearly shows how much data is used by showing in visualization as light and dark color. • In this task correlation matrix for five numerical columns is printed. • Columns are: 'replies_count', 'participants count', 'likes', 'comment' and 'shares'. • Visualization of correlation matrix is generated by using heatmap (it uses seaborn library). • This visualization is performed to show the relation between these engagements. 

		<table><caption>Correlation Matrix of Engagement Metrics</caption><tr><th></th><th>replies_count</th><th>participants_count</th><th>likes</th><th>comments</th><th>shares</th></tr><tr><th>replies_count</th><td>1</td><td>0.97</td><td>-0.019</td><td>-0.0066</td><td>-0.019</td></tr><tr><th>participants_count</th><td>0.97</td><td>1</td><td>-0.02</td><td>-0.0084</td><td>-0.02</td></tr><tr><th>likes</th><td>-0.019</td><td>-0.02</td><td>1</td><td>0.16</td><td>1</td></tr><tr><th>comments</th><td>-0.0066</td><td>-0.0084</td><td>0.16</td><td>1</td><td>0.16</td></tr><tr><th>shares</th><td>-0.019</td><td>-0.02</td><td>1</td><td>0.16</td><td>1</td></tr></table>		replies_count	participants_count	likes	comments	shares	replies_count	1	0.97	-0.019	-0.0066	-0.019	participants_count	0.97	1	-0.02	-0.0084	-0.02	likes	-0.019	-0.02	1	0.16	1	comments	-0.0066	-0.0084	0.16	1	0.16	shares	-0.019	-0.02	1	0.16	1
	replies_count	participants_count	likes	comments	shares																																	
replies_count	1	0.97	-0.019	-0.0066	-0.019																																	
participants_count	0.97	1	-0.02	-0.0084	-0.02																																	
likes	-0.019	-0.02	1	0.16	1																																	
comments	-0.0066	-0.0084	0.16	1	0.16																																	
shares	-0.019	-0.02	1	0.16	1																																	
		Analysis and Result: • High Positive Correlation a. Likes vs shares (1): shows perfect correlation of 1. This shows red flag in terms of data science. This data might contain identical data or that one is the direct directives of others in this specific data set. b. Replies_count vs participants_count (0.97): This part shows strong positive correlation. This indicates that the number of participants in a discussion increases the total number of replies increases almost linearly. • No Correlation: a. Likes/shares vs replies count (-0.0019): This shows almost no linear relationship between likes/shares and replies count. b. Comments vs likes/shares (0.16): There is a weak positive correlation. While the tendency of post is more likely to have more comments, these relationships do not show that type of predictivity.																																				
5	For the discrete/categorical columns 'language' and	Step5:																																				

'author', find out how many values are 'null' or 'empty'. Apply a data imputation technique of your choice to replace missing values in any one of the three columns ('language' and 'author')

In this task I have identified the missing and null empty values in the 'author' and the 'language' columns and applied the imputation techniques. In this part I found that there is too many missing authors which is analyzed and fixed below.

- For author the code replaces the missing values with a placeholder string that is 'unknown_author'.
- For language the uses mode imputation, replacing missing values with the most frequent language found in the dataset (df['language'].mode()[0]).

```
Categorical Missing Values Implementation for Language and Author

1 cat_cols = ['language', 'author']
2 ✓ [18] < 10 ms

Printing Null and Empty Values

1 nulls = df[cat_cols].isnull().sum()
2 empties = (df[cat_cols] == '').sum()
3 ✓ [19] < 10 ms

1 print("Missing Null and Empty for author and language:")
2 print(nulls + empties)
3 ✓ [27] < 10 ms

Missing Null and Empty for author and language:
language      0
author      2467
dtype: int64
```

- Replace all the empty strings of the author and language using .replce('',np.nan) python script.
- After replacing the strings, I proceeded for filling the strings using .fillna('unknown_author) python script. Filled null or empty strings as unknown_author.

Replacing Empty strings with NaN

```
1 df["author"] = df["author"].replce('',np.nan)
2 ✓ [46] < 10 ms
```

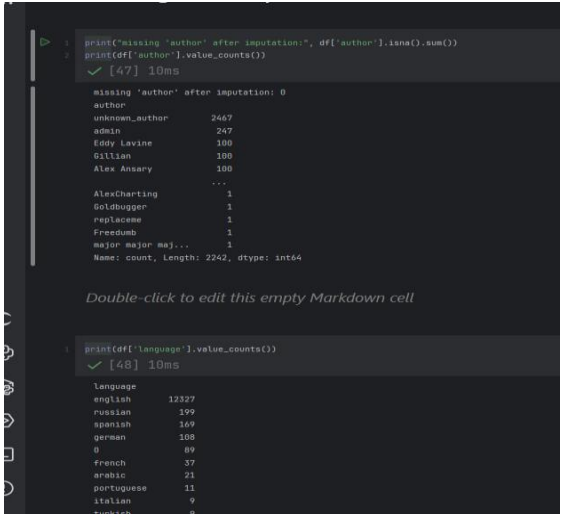
Filling Author and Language using fillna

```
1 df["author"] = df["author"].fillna("unknown_author")
2 ✓ [44] < 10 ms
```

Checking Missing Values:

```
1 print("Missing 'author' after imputation:", df['author'].isna().sum())
2 ✓ [45] < 10 ms

Missing 'author' after imputation: 0
```

		After finding the empty and null values I proceed for filling the authors using fillna and after filling printed the null value that is 0. Finally, imputation done.  <pre> print('missing author' after imputation:', df['author'].isna().sum()) print(df['author'].value_counts()) [47] 10ms missing 'author' after imputation: 0 author unknown_author 2467 admin 247 Edy Levine 100 Gillian 100 Alex Ansary 100 ... AlexCherling 1 Spidbopper 1 replaceme 1 Freedom 1 major major maj... 1 Name: count, Length: 2242, dtype: int64 Double-click to edit this empty Markdown cell print(df['language'].value_counts()) [48] 10ms language english 12327 russian 199 spanish 169 german 108 0 89 french 37 arabic 21 portuguese 11 italian 9 turkish 9 </pre> Analysis and Result: • Before imputation it is found that there is 2467 authors null and empty. • In case of language there are not any null and missing values in the language. • So, I did imputation for author only. • This code correctly and completely replaces empty strings ("") with nan before performing the count, ensures that 'empty values cannot be accidently skipped by the .isnull() checking. • After checking it is found that there missing null and empty values in the author column is 2467 whereas the missing values for language is 0.
	Total	

Task 2

	Task	Comments
1	Investigate and conclude whether there is evidence of a flooding attack in the file. If yes, what is the type of the attack and how many flooding packets were found out of total packets. For example, File 1 – No flooding attack – 0/500 flooding packets File 2 – SYN flood – 421/500 flooding packets	This is type of network attack analysis. In this task, pcap and pcapng files are analyzed if there are any types of attack exists or not in that file. I performed these files analysis by using python script and verified the analysis is correct or not inside the Wireshark. To begin analysis first of all, I have imported the following libraries of python. • Scapy , rdcap, • Plotly • Pandas Step1: • The analysis of .pcapng and .pcap file is analyzed using python-based approach with the scapy library. • Each packet was analyzed to detect three common types of attacks. a. Syn attack b. Ping attack c. Snmp-udp attack For analysis of these types of attack I have defined a function def analyze_floods(pkts)

Defining function for Flood Analysis

```
1 def analyze_floods(pkts):
2     syn = ping = snmp = 0
3     proto_count = {"TCP": 0, "UDP": 0, "ICMP": 0, "Other": 0}
4
5     for pkt in pkts:
6         if pkt.haslayer(TCP):
7             proto_count["TCP"] += 1
8             flags = pkt[TCP].flags
9             # SYN = 1, ACK = 0
10            if flags & 0x02 and not flags & 0x10:
11                syn += 1
12
13        elif pkt.haslayer(UDP):
14            proto_count["UDP"] += 1
15            if pkt[UDP].sport == 161 or pkt[UDP].dport == 161:
16                snmp += 1
17
18        elif pkt.haslayer(ICMP):
19            proto_count["ICMP"] += 1
20            if pkt[ICMP].type == 8:
21                ping += 1
22
23        else:
24            proto_count["Other"] += 1
25
26    total_packets = len(pkts)
27    return syn, ping, snmp, total_packets, proto_count
28
```

✓ [5] < 10 ms

File 1.pcapng:

This file is loaded using python script:

Loading file 1.pcapng and printing the total packets

```
1 file_1 = "File 1.pcapng"
2 packets_1 = rdpcap(file_1)
3 print(f"{file_1} loaded with {len(packets_1)} packets")
4
```

✓ [7] 186ms

File 1.pcapng loaded with 896 packets

After loading total number of packets inside this file is also printed:

Total packets = 896

Then after that I have classified the types of flooding attack.

Classifying the Types of Flooding Attack

```
syn_1, ping_1, snmp_1, total_1, proto_count_1 = analyze_floods(packets_1)
flood_total_1 = syn_1 + ping_1 + snmp_1
flood_counts_1 = {"SYN Flood": syn_1, "Ping Flood": ping_1, "SNMP Flood": snmp_1}
attack_type_1 = "No any type of Flooding attack detected"
if flood_total_1 > 0:
    attack_type_1 = max(flood_counts_1, key=flood_counts_1.get)
✓ [8] 19ms
```

--	--

- | | |
|--|--|
| | |
|--|--|

--	--

[illegible]

--	--

- | | |
|--|--|
| | |
|--|--|

--	--

- It is found that Multiple external source address sending SYN packets whereas the is not any returned ack from the destination Ips.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.100	10.0.2.15	IP	60	60 bytes to 10.0.2.15
2	0.000000	192.168.1.100	10.0.2.15	TCP	60	60 bytes to 10.0.2.15
3	0.000000	192.168.1.100	10.0.2.15	IP	60	60 bytes to 10.0.2.15
4	0.000000	192.168.1.100	10.0.2.15	TCP	60	60 bytes to 10.0.2.15
5	0.000000	192.168.1.100	10.0.2.15	IP	60	60 bytes to 10.0.2.15
6	0.000000	192.168.1.100	10.0.2.15	TCP	60	60 bytes to 10.0.2.15
7	0.000000	192.168.1.100	10.0.2.15	IP	60	60 bytes to 10.0.2.15
8	0.000000	192.168.1.100	10.0.2.15	TCP	60	60 bytes to 10.0.2.15
9	0.000000	192.168.1.100	10.0.2.15	IP	60	60 bytes to 10.0.2.15
10	0.000000	192.168.1.100	10.0.2.15	TCP	60	60 bytes to 10.0.2.15

- Inside hierarchy statistics it is found that there is high volume of SYN packets that are sent without 0 ack. The packets' direction is 100% from source to destination and 0% from destination to source.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	POUs
Frame	100.0	354	100.0	25130	245	0	0	0	354
Ethernet	100.0	354	20.7	5202	50	0	0	0	354
Internet Protocol Version 4	100.0	354	28.2	7080	69	0	0	0	354
Transmission Control Protocol	100.0	354	91.1	12848	125	354	12848	125	354

Step2:

File 2.pcap

For analysis of file 2.pcap I have loaded this file, I follow the steps as I did for file 1 and also classified the type of attack.

Loading file 2.pcap for analysis

```
1 file_2 = "File 2.pcap"
2 packets_2 = rdpcap(file_2)
3 print(f'{file_2} is loaded with {len(packets_2)} packets')
4 [13] 3s 19ms
5 File 2.pcap is loaded with 10000 packets
```

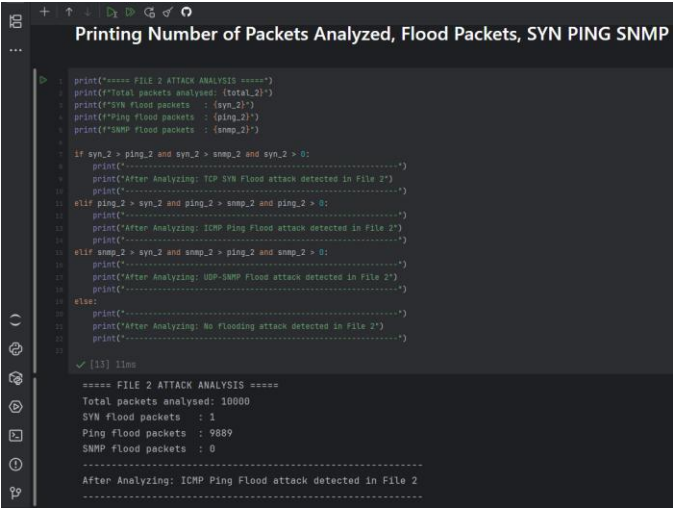
Classifying the type of Attack

```
1 syn_2, ping_2, snmp_2, total_2, proto_count_2 = analyze_floods(packets_2)
2 flood_total_2 = syn_2 + ping_2 + snmp_2
3 [14] 169ms
4 Flood_counts_2 = {"Total SYN Flood": syn_2, "Total Ping Flood": ping_2, "Total SNMP Flood": snmp_2}
5 attack_type_2 = "There is No any type of flooding attack"
6 if flood_total_2 > 0:
7     attack_type_2 = max(flood_counts_2, key=flood_counts_2.get)
8 [15] < 10 ms
```

After examining file 2.pcap flooding packets counts it is found that the dominant attack type was ICMP ping flood attack was identified.

Artifacts of this analysis:

- Total packets present in this file: 10000
- SYN flooding packets: 1
- Ping flooding packets: 9889
- There are not any SNMP flooding packets.
- There is ICMP flood attack that has been detected.



```
Printing Number of Packets Analyzed, Flood Packets, SYN PING SNMP

1 print("==== FILE 2 ATTACK ANALYSIS =====")
2 print("Total packets analysed: (total2)")
3 print("SYN flood packets : (syn_2)")
4 print("Ping flood packets : (ping_2)")
5 print("SNMP flood packets : (snmp_2)")
6
7 if syn_2 > ping_2 and syn_2 > snmp_2 and syn_2 > 0:
8     print(".....")
9     print("After Analyzing: TCP SYN Flood attack detected in File 2")
10    print(".....")
11 elif ping_2 > syn_2 and ping_2 > snmp_2 and ping_2 > 0:
12    print(".....")
13    print("After Analyzing: ICMP Ping Flood attack detected in File 2")
14    print(".....")
15 elif snmp_2 > syn_2 and snmp_2 > ping_2 and snmp_2 > 0:
16    print(".....")
17    print("After Analyzing: UDP-SNMP Flood attack detected in File 2")
18    print(".....")
19 else:
20    print(".....")
21    print("After Analyzing: No flooding attack detected in File 2")
22    print(".....")
23
24 [13] lina
===== FILE 2 ATTACK ANALYSIS =====
Total packets analysed: 10000
SYN flood packets : 1
Ping flood packets : 9889
SNMP flood packets : 0
.....
After Analyzing: ICMP Ping Flood attack detected in File 2
.....
```

Analysis in Wireshark:

- File 2.pcap is loaded in Wireshark and applied filter icmp.type == 8.
- After applying this filter it is found that all packets are directed towards a single destination and source ip address are highly varied as distributed attack or spoofing.
- While analyzing the timestamp dozens of packets are being sent within a fraction of millisecond.

No.	Time	Source	Destination	Protocol	Length	Info
1118	0.879895	81.168.16.168	192.168.1.1	ICMP	55	Echo (ping) request [Echo(78), seq=78081645, ttl=64 (no response found)]
1119	0.879902	192.168.1.1	192.168.1.1	ICMP	56	Echo (ping) request [Echo(78), seq=78081645, ttl=64 (no response found)]
1120	0.879904	150.182.143.216	192.168.1.1	ICMP	84	Echo (ping) request [Echo(78), seq=4319786789, ttl=79 (no response found)]
1121	0.879906	192.168.1.1	192.168.1.1	ICMP	80	Echo (ping) request [Echo(78), seq=4319786789, ttl=79 (no response found)]
1122	0.879908	56.187.177.112	192.168.1.1	ICMP	316	Echo (ping) request [Echo(78), seq=4442771575, ttl=76 (no response found)]
1123	0.879910	216.167.212.17	192.168.1.1	ICMP	136	Echo (ping) request [Echo(78), seq=183051646, ttl=64 (no response found)]
1124	0.879912	41.55.161.221	192.168.1.1	ICMP	141	Echo (ping) request [Echo(78), seq=20037754697, ttl=148 (no response found)]
1125	0.879914	141.78.42.133	192.168.1.1	ICMP	519	Echo (ping) request [Echo(78), seq=1748764123, ttl=109 (no response found)]
1126	0.879916	180.156.244.79	192.168.1.1	ICMP	1218	Echo (ping) request [Echo(78), seq=12180721086, ttl=182 (no response found)]
1127	0.879918	151.203.223.164	192.168.1.1	ICMP	918	Echo (ping) request [Echo(78), seq=39387751737, ttl=86 (no response found)]
1128	0.879920	94.52.128.110	192.168.1.1	ICMP	105	Echo (ping) request [Echo(78), seq=13163712186, ttl=160 (no response found)]
1129	0.879922	190.81.82.239	192.168.1.1	ICMP	68	Echo (ping) request [Echo(78), seq=51219763439, ttl=111 (no response found)]
1130	0.879924	42.284.230.69	192.168.1.1	ICMP	522	Echo (ping) request [Echo(78), seq=20588723657, ttl=79 (no response found)]
1131	0.879926	190.68.221.82	192.168.1.1	ICMP	612	Echo (ping) request [Echo(78), seq=19674758884, ttl=122 (no response found)]
1132	0.879928	170.54.114.118	192.168.1.1	ICMP	1011	Echo (ping) request [Echo(78), seq=10774762125, ttl=148 (no response found)]
1133	0.879930	62.172.135.36	192.168.1.1	ICMP	1127	Echo (ping) request [Echo(78), seq=12128734863, ttl=151 (no response found)]
1134	0.879932	218.16.12.71	192.168.1.1	ICMP	1076	Echo (ping) request [Echo(78), seq=30385723526, ttl=144 (no response found)]
1135	0.879934	4.18.162.183	192.168.1.1	ICMP	64	Echo (ping) request [Echo(78), seq=10613971515, ttl=162 (no response found)]
1136	0.879936	4.18.162.183	192.168.1.1	ICMP	1309	Echo (ping) request [Echo(78), seq=10613971515, ttl=93 (no response found)]
1137	0.879938	166.167.165.178	192.168.1.1	ICMP	535	Echo (ping) request [Echo(78), seq=41816717176, ttl=156 (no response found)]
1138	0.879940	150.41.113.98	192.168.1.1	ICMP	1364	Echo (ping) request [Echo(78), seq=13951496188, ttl=110 (no response found)]
1139	0.879942	228.5.212.161	192.168.1.1	ICMP	1240	Echo (ping) request [Echo(78), seq=48776758895, ttl=88 (no response found)]

- Protocol hierarchy statistics total packets were 9889 packets and 100% is ICMP.

This observation concludes that there is ICMP flood attack, and type of both DoS and DDoS attack.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	9889	100.0	7652524	156 M	0	0	0	9889
Ethernet	100.0	9889	1.8	138446	2832 k	0	0	0	9889
Internet Protocol Version 4	100.0	9889	2.6	197780	4046 k	0	0	0	9889
Internet Control Message Protocol	100.0	9889	95.6	7316298	149 M	8535	6303675	128 M	9889
HiperConTracer Trace Protocol	13.7	1354	13.1	1001791	20 M	1354	1001791	20 M	1354

File 3.pcapng:

for this task followed the same steps as above for loading and classifying the flooding attack.

```

Analysis of file 3

Loading File 3.pcapng

file_3 = "File 3.pcapng"
packets_3 = rdpcap(file_3)
print(f"File 3 is loaded with {len(packets_3)} packets")
[20] 8s 727ms

File 3.pcapng is loaded with 4373 packets

Classifying the type of attack

syn_3, ping_3, snmp_3, total_3, proto_count_3 = analyze_floods(packets_3)
flood_total_3 = syn_3 + ping_3 + snmp_3
[21] 100ms

flood_counts_3 = {"SYN Flood": syn_3, "Ping Flood": ping_3, "SNMP Flood": snmp_3}
attack_type_3 = "No flooding attack detected"
if flood_total_3 > 0:
    attack_type_3 = max(flood_counts_3, key=flood_counts_3.get)
[22] < 10 ms
  
```

In this file there is SNMP flooding type of attack is detected.

- Total number of packets is 4373.
- Total syn packets =0, ping packets =0.
- Total SNMP flood packets = 4079

- There is SNMP flooding packets detected.

Printing The Type of Attack

```

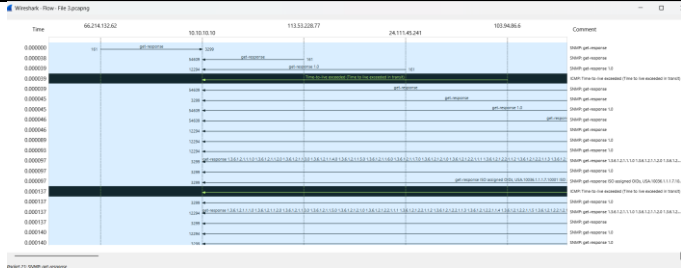
1  ## Printing The Type of Attack (Corrected)
2  print("==== FILE 3 ATTACK ANALYSIS ====")
3  print(f"Total number of packets analysed: {total_3}")
4  print(f"Total SYN Flood packets : {syn_3}")
5  print(f"Total Ping Flood packets : {ping_3}")
6  print(f"Total SNMP Flood packets : {snmp_3}")
7
8  if syn_3 > ping_3 and syn_3 > snmp_3 and syn_3 > 0:
9      print("-----")
10     print("After Analyzing: TCP SYN Flood attack detected in File 3")
11     print("-----")
12 elif ping_3 > syn_3 and ping_3 > snmp_3 and ping_3 > 0:
13     print("-----")
14     print("After Analyzing: ICMP Ping Flood attack detected in File 3")
15     print("-----")
16 elif snmp_3 > syn_3 and snmp_3 > ping_3 and snmp_3 > 0:
17     print("-----")
18     print("After Analyzing: UDP-SNMP Flood attack detected in File 3")
19     print("-----")
20 else:
21     print("-----")
22     print("After Analyzing: No flooding attack detected in File 3")
23     print("-----")
24
25 ✓ [23] 17ms

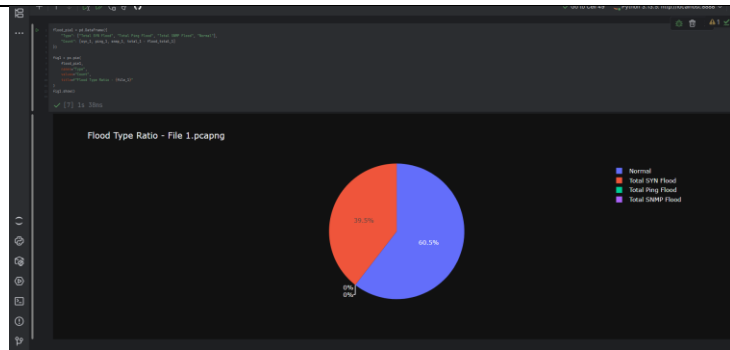
```

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help							
							
tcp.flags.syn == 1 && tcp.flags.ack == 0							
No.	Time	Source	Destination	Protocol	Length	Info	

- While analyzing the protocol hierarchy it is found that all the captured frames are ethernet With IPV4 traffic.
- the most traffic uses UDP (user datagram protocol) that is 4079 (93.3%) packets.
- this strongly indicates that file 3 have SNMP flood attack.

Wireshark - Protocol Hierarchy Statistics - File 3.pcapng									
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDOs
Frame	100.0	4373	100.0	1055847	359 M	0	0	0	4373
Ethernet	100.0	4373	5.8	61222	20 M	0	0	0	4373
Internet Protocol Version 4	100.0	4373	8.3	87460	29 M	0	0	0	4373
User Datagram Protocol	93.3	4079	3.1	32632	11 M	0	0	0	4079
Simple Network Management Protocol	93.3	4079	81.1	856060	291 M	4079	856060	291 M	4079
Internet Control Message Protocol	6.7	294	1.7	18473	6289 k	294	18473	6289 k	294

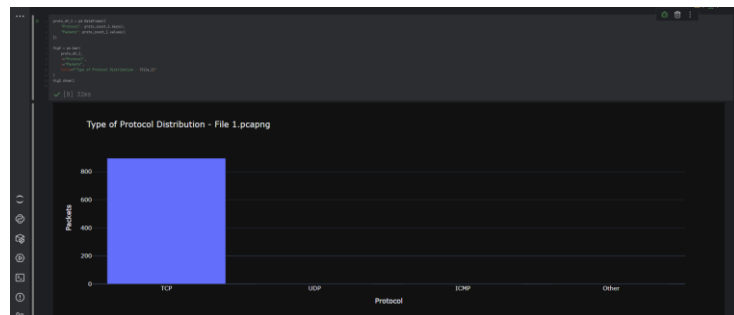
		
2	In files where there is evidence of a flooding attack, present your results visually, as follows, - A pie chart to show the ratio of possible flood attacks. - A bar chart to show the distribution of packet types by protocol in the file.	Visualization of File 1.pcapng Final analysis of this file is represented using two different types of charts, that is pie chart and bar chart containing different types of data. Pie-chart → Flood Type Ratio This chart shows the proportion of different traffic categories within the file. • Blue: It is dominating part in the given chart holding 60.5%, which indicates most of the captured packets. This part shows no type of attack. • Red: This is second largest part of the chart showing SYN attack detected, it covers 39.5% of the chart and it is primary part of attack. • Green/Purple: Since this color is invisible in the chart with 0%, so there is not any SNMP and ping flood attack.



Bar Graph:

It completely focusses on the total volume of packets categorized by the communication protocol.

- The large blue bar confirms that the TCP protocol which is 896 packets and aligns with the SYN flood attack.
- As SYN is the TCP based flag.



File 2.pcap

Visual Analysis of File 2:

Pie Chart:

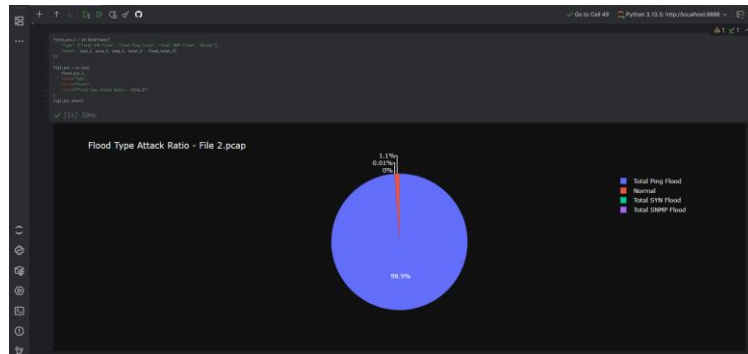
The pie chart shows the aggressive type of volumetric attack type of ping.

- **Blue:**
Dominating part of the chart consists about 98.9% of the total part as ping flood attack type.
- **Red:**
Normal Traffic consists only 1.1%
- **Green:**

Total syn and SYN type of flood attack that covers only 0.01% of the total part.

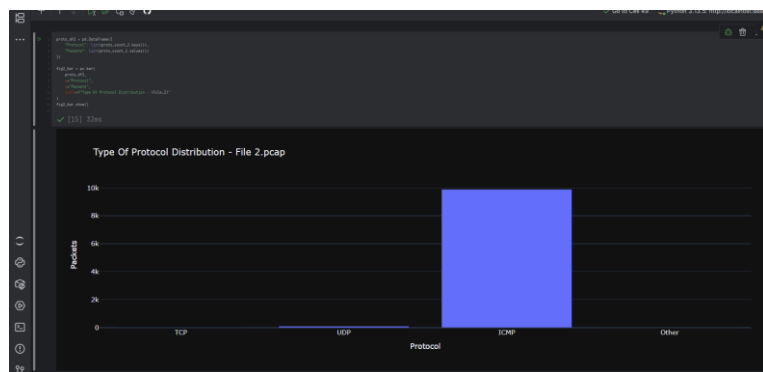
- **Purple:**

This shows 0% of the total part which is SNMP flood attack.



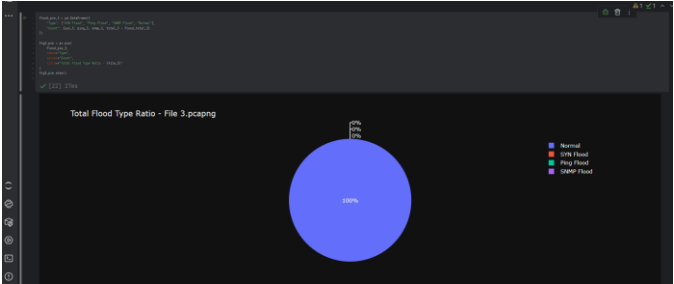
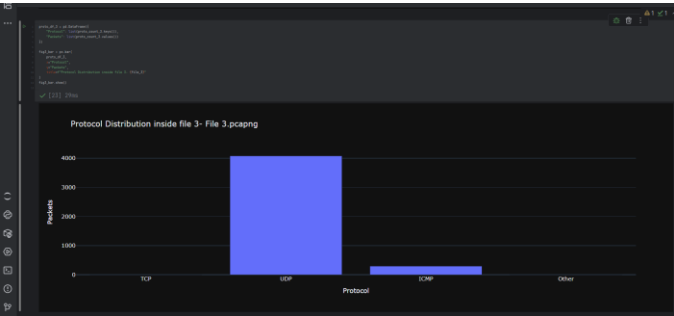
Bar chart file 2:

- The bar chart shows a large spike(volume) of ICMP which is about 10,000.
- In the UDP part there is a small fraction of traffic.
- TCP and others are at 0%.
- It means traffic is completely drown out by ICMP Ping flood attack.



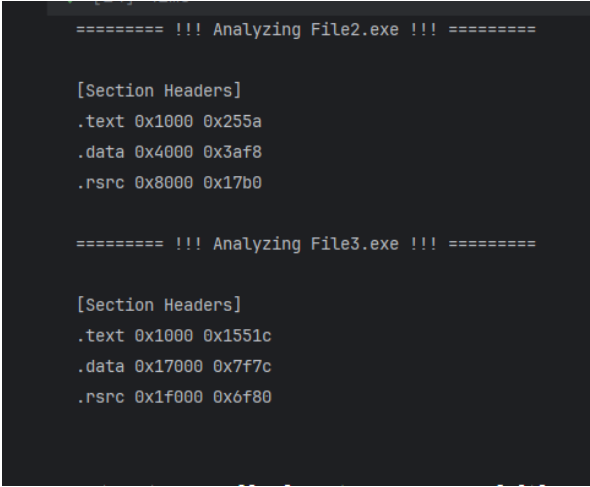
File 3.pcap visual analysis:

- The pie chart shows the total flood ratio.
- **Blue:**

		This part covers maximum part of the chart that is 93.3% of the total part of the chart. Showing that SNMP flood attack. Red: It consists of 6.72% of the total part pointing that there is not any type of attack. It shows the Normal part. Purple/Green: It is invisible in the chart that means there is no any type of SNMP and Ping flood type of attack.  File 2 Bar Chart Visualization: - In this chart the traffic volume is concentrated at the UDP protocol portion which is 4079 packets. - At the ICMP protocol portion the traffic is very low, which is about 294 only - At TCP and other there is no any type of traffic without any activity. 
	Total	

Task 3

	Task	Comments
1	Load the files using a Python script.	This task concludes about the analysis of the malicious activity of .exe files. To perform analysis following libraries are imported: • Pefile • Re • Sys, etc Step1: Loaded all the targeted exe files in the jupyter notebook using python script. All these files were processed individually to ensure the accurate and independent analysis of all the files. Loading ALL EXE files for analysis <pre>all_files = ["File1.exe", "File2.exe", "File3.exe"] [201]</pre>
2	Print all the Section headers.	Step2: • In this code all section headers are extracted and displayed, and this includes the section name, virtual address, virtual size. • This section headers helps to analyse the internal layout and abnormal of suspicious sections often that is used by the malware. More tool windows <pre>for file_path in all_files: print(f'\n===== !!! Analyzing {file_path} !!! =====') try: pe = pefile.PE(file_path) print('\n[Section Headers]') for section in pe.sections: print(section.Name.decode(errors='ignore').rstrip('\x00'), hex(section.VirtualAddress), hex(section.Misc_VirtualSize)) except pefile.PEFormatError: print(f'{file_path} is not a valid PE file no any Section Headers') [14] 41ms</pre> <pre>===== !!! Analyzing File1.exe !!! =====</pre> <pre>[Section Headers] .text 0x1000 0x126b0 .data 0x14000 0x101c .rsrc 0x16000 0x8960</pre>

		 <pre> ===== !!! Analyzing File2.exe !!! ===== [Section Headers] .text 0x1000 0x255a .data 0x4000 0x3af8 .rsrc 0x8000 0x17b0 ===== !!! Analyzing File3.exe !!! ===== [Section Headers] .text 0x1000 0x1551c .data 0x17000 0x7f7c .rsrc 0x1f000 0x6f80 </pre> After analysing section header we got that all the section headers of all three file are same. • .text • .data • .rsrc
3	Print all the imported libraries.	Step3: • The imported library sections show the dynamic Link libraries (DLLs) and shows the corresponding functions of the files. • This script clarifies all the DLLs of the files. • Imported functions provides insights of the behaviour of the executables files such as memory manipulation, process injection, or network activity. • There are many imported libraries found inside these files.

```
Task_3.ipynb x dashboard.py
+ | ↑ | ↓ | ⏏ | ↺ | ⏏ | ⏏
... ▶ 1 for file_path in all_files:
2     print(f"\n===== !!! Imported Library in {file_path} !!! =====")
3     try:
4         pe = pefile.PE(file_path)
5
6         if hasattr(pe, "DIRECTORY_ENTRY_IMPORT"):
7             for entry in pe.DIRECTORY_ENTRY_IMPORT:
8                 print(f"\nDLL: {entry.dll.decode(errors='ignore')}")
9                 for imp in entry.imports:
10                    if imp.name:
11                        print(f"    {imp.name.decode(errors='ignore')}")
12                    else:
13                        print(f"    ordinal_{imp.ordinal}")
14            else:
15                print("No any imports found in this file")
16
17        except pefile.PEFormatError:
18            print("given file is Invalid PE file")
19
✓ [15] 44ms

===== !!! Imported Library in File1.exe !!! =====

DLL: SHELL32.dll
    ShellAboutW

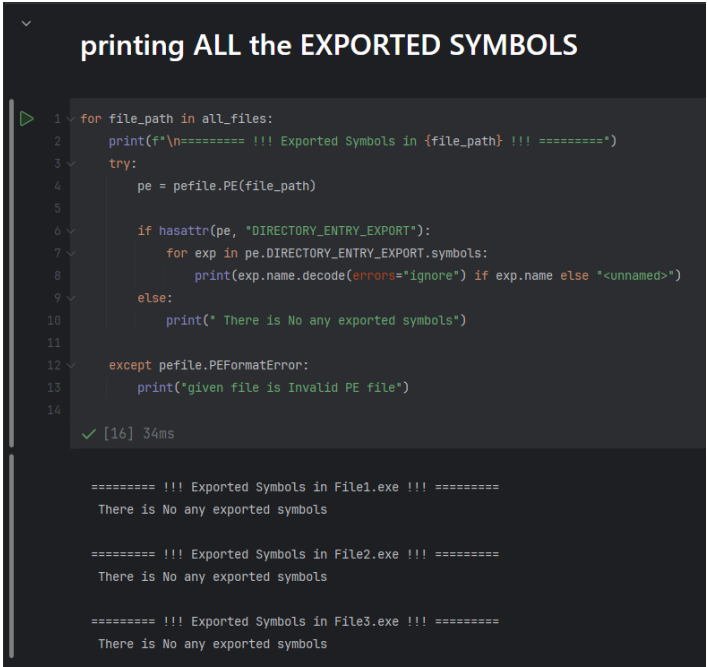
DLL: msvcrt.dll
    __CxxFrameHandler
    _CxxThrowException
    wcstoul
    toupper
    wcschr
```

```
===== !!! Imported Library in File2.exe !!! =====

DLL: USER32.dll
    CharToOemW

DLL: SHELL32.dll
    CommandLineToArgvW

DLL: msvcrt.dll
    _controlfp
    __set_app_type
    __p__fmode
    __p__commode
```

			<pre> ===== !!! Imported Library in File3.exe !!! ===== DLL: msvcrt.dll __p__commode __adjust_fdiv __setusermatherr __initterm __p__fmode __acmdln exit _cexit __set_app_type </pre>	
4	Print all the exported symbols.	Step4: • In this section all the exported symbols are analysed. • All the exported symbols are extracted for identification of functions exposed by the executable (.exe) file. • After analysing these files there is not any export symbols are found.	 <pre> printing ALL the EXPORTED SYMBOLS 1 for file_path in all_files: 2 print(f"\n===== !!! Exported Symbols in {file_path} !!! =====") 3 try: 4 pe = pefile.PE(file_path) 5 6 if hasattr(pe, "DIRECTORY_ENTRY_EXPORT"): 7 for exp in pe.DIRECTORY_ENTRY_EXPORT.symbols: 8 print(exp.name.decode(errors="ignore") if exp.name else "<unnamed>") 9 else: 10 print("There is No any exported symbols") 11 12 except pefile.PEFormatError: 13 print("given file is Invalid PE file") 14 15 [16] 34ms ===== !!! Exported Symbols in File1.exe !!! ===== There is No any exported symbols ===== !!! Exported Symbols in File2.exe !!! ===== There is No any exported symbols ===== !!! Exported Symbols in File3.exe !!! ===== There is No any exported symbols </pre>	
5	Print all the string features of length four or more characters.	Step5: • In this part string features are extracted which is printed in the ASCII strings of length four or more characters.		

- Strings are responsible for revealing hardcoded URLs or IP address, command strings, file paths, and suspicious keywords.
- It also helps for identifying hidden or embedded malicious indicators without executing the file.

```

1  for file_path in all_files:
2      print(f"\n===== !!! Strings in {file_path} !!! =====")
3      with open(file_path, "rb") as f:
4          data = f.read()
5          strings = re.findall(rb"[ -~]{4,}", data)
6
7          for s in strings[:10]: # show first 10
8              print(s.decode(errors="ignore"))
9
✓ [17] 10ms
===== !!! Strings in File1.exe !!! =====
!This program cannot be run in DOS mode.
$X79
$y70$X79
$X7T
$X79
$X7Rich
.text
.data
.rsrc
SHELL32.dll

```

```

===== !!! Strings in File2.exe !!! =====
!This program cannot be run in DOS mode.
BRich
.text
.data
.rsrc
USER32.dll
SHELL32.dll
msvcrt.dll
KERNEL32.dll
NTDLL.DLL

```


		<pre> ===== !!! Strings in File3.exe !!! ===== !This program cannot be run in DOS mode. Rich .text .data .rsrc msvcrt.dll ADVAPI32.dll KERNEL32.dll NTDLL.DLL GDI32.dll </pre>
6	Create a list of following suspicious imports ('GetProcAddress' 'CreateRemoteThrea d', 'TerminateProcess', 'VirtualAlloc', and 'WriteProcessMemor y') and write a script to check if the calls exists in a sample file. If a file has more than 50% matches, it should be considered malicious.	Step6: • In this section all the suspicious imports is list that is asked to do in the question is listed. • These suspicious list of windows API functions commonly associated with the malicious activity. Defining Suspicious Imports <pre> 1 SUSPICIOUS_IMPORTS = [2 'GetProcAddress', 3 'CreateRemoteThread', 4 'TerminateProcess', 5 'VirtualAlloc', 6 'WriteProcessMemory' 7] [207] </pre> • For the analysis of the suspicious imports, I checked that, if these suspicious API call exists in all the flies or not. • The after that I have removed all the duplicates. • Then after that inside this code suspicious imports percentage was calculated. Suspicious Percentage = $\frac{\text{Number of Detected imports}}{\text{Total Suspicious imports}} \times 100$

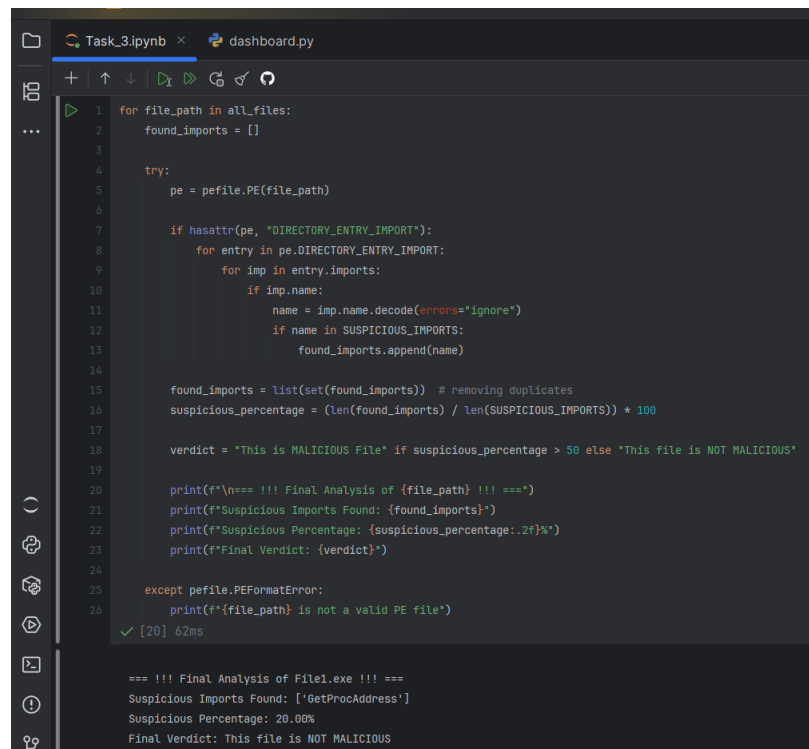
- According to the malware criteria given by the question, if file contains more than 50% of suspicious imports, then it is classified as malicious otherwise Not malicious.

Analysis Results:

There are not any files are malicious because:

- File 1.exe have only 20% imports.
- File 2.exe have 0% imports.
- File 3.exe have 40% imports.

All these files have less than 50% so it is considered as not malicious.



```

1 for file_path in all_files:
2     found_imports = []
3
4     try:
5         pe = pefile.PE(file_path)
6
7         if hasattr(pe, "DIRECTORY_ENTRY_IMPORT"):
8             for entry in pe.DIRECTORY_ENTRY_IMPORT:
9                 for imp in entry.imports:
10                     if imp.name:
11                         name = imp.name.decode(errors="ignore")
12                         if name in SUSPICIOUS_IMPORTS:
13                             found_imports.append(name)
14
15     found_imports = list(set(found_imports)) # removing duplicates
16     suspicious_percentage = (len(found_imports) / len(SUSPICIOUS_IMPORTS)) * 100
17
18     verdict = "This is MALICIOUS File" if suspicious_percentage > 50 else "This file is NOT MALICIOUS"
19
20     print(f"\n=== !!! Final Analysis of {file_path} !!! ===")
21     print(f"Suspicious Imports Found: {found_imports}")
22     print(f"Suspicious Percentage: {suspicious_percentage:.2f}%")
23     print(f"Final Verdict: {verdict}")
24
25 except pefile.PEFormatError:
26     print(f"{file_path} is not a valid PE file")

```

✓ [20] 62ms

```

=== !!! Final Analysis of File1.exe !!! ===
Suspicious Imports Found: ['GetProcAddress']
Suspicious Percentage: 20.00%
Final Verdict: This file is NOT MALICIOUS

```

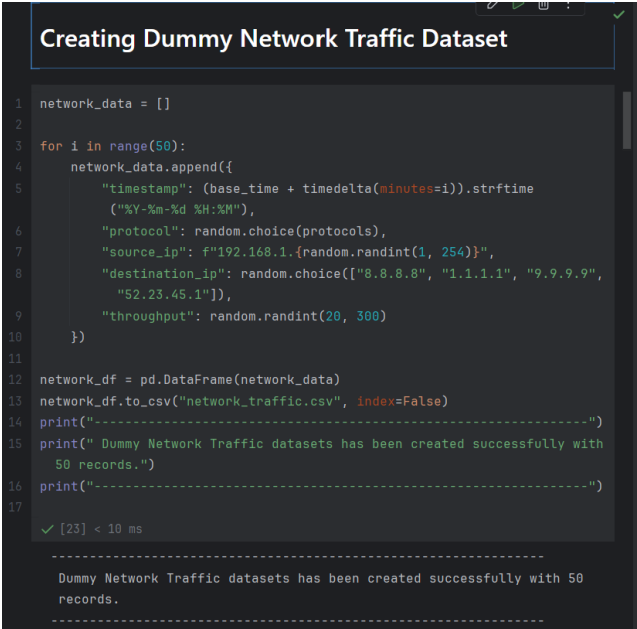
Final Analysing file 1.exe:

- After running the python scripts in loop for final verdict of the malicious and suspicious imports checking. Found that file 1 consists of GetProcAddress as suspicious imports which is only 20% according to the question a file to be malicious suspicious percentage should be more than 50%. So, file 1 is not malicious.

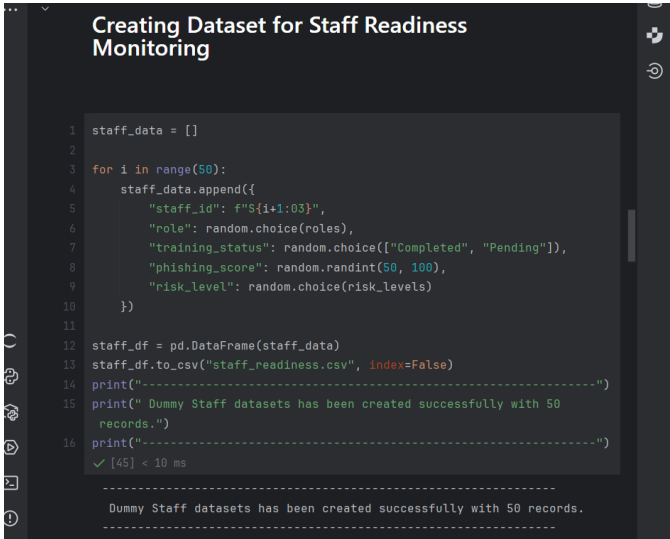
		<pre> === !!! Final Analysis of File2.exe !!! === Suspicious Imports Found: [] Suspicious Percentage: 0.00% Final Verdict: This file is NOT MALICIOUS === !!! Final Analysis of File3.exe !!! === Suspicious Imports Found: ['TerminateProcess', 'GetProcAddress'] Suspicious Percentage: 40.00% Final Verdict: This file is NOT MALICIOUS </pre> Final Analysis of file 2.exe - In this file no types of suspicious is found and suspicious percentage is 0%. So, it is also not a malicious file. Analysis of file 3.exe - File 3 has two suspicious imports 'TerminateProcess and GetProcAddress' the percentage consistency of suspicious is 40%. - According to the demand of the question this file is not a malicious file.
	Total	

Task 4

	Task	Comments
1	Data collection advise for network traffic monitoring.	Step1: - By using python code, I have created dummy data for task 4 assessment. - The code shown in the screenshot stimulates the network traffic data. - It generates timestamp record which includes protocols type, source and destination Ip addresses, and throughout the values. - The given attribute of this code stimulates the major data, which is necessary for network traffic analysis or monitoring, that allows detection of abnormal traffic volumes, suspicious

		protocols and potential network-based attacks such as flooding or scanning.  <pre> 1 network_data = [] 2 3 for i in range(50): 4 network_data.append({ 5 "timestamp": (base_time + timedelta(minutes=i)).strftime 6 ("%Y-%m-%d %H:%M"), 7 "protocol": random.choice(protocols), 8 "source_ip": f"192.168.1.{random.randint(1, 254)}", 9 "destination_ip": random.choice(["8.8.8.8", "1.1.1.1", "9.9.9.9", 10 "52.23.45.1"]), 11 "throughput": random.randint(20, 300) 12 }) 13 14 network_df = pd.DataFrame(network_data) 15 network_df.to_csv("network_traffic.csv", index=False) 16 print("-----") 17 print(" Dummy Network Traffic datasets has been created successfully with 18 50 records.") 19 print("-----") </pre> ✓ [23] < 10 ms ----- Dummy Network Traffic datasets has been created successfully with 50 records. -----
2	Data collection advise for endpoints monitoring.	Step2: • As above this code creates endpoint security dataset which represents any individual devices of an organization. • It is responsible for capturing operating system versions, firewalls status, and malware alert counts. • Endpoint dataset paly major role for monitoring endpoint, enabling identification of vulnerable systems, disable security controls and compromised devices.

		Creating Dummy Endpoint Dataset <pre> 1 endpoint_data = [] 2 3 for i in range(50): 4 endpoint_data.append({ 5 "device": f"PC-{i+1:02}", 6 "os_version": random.choice(os_versions), 7 "firewall_status": random.choice(firewall_status), 8 "malware_alerts": random.randint(0, 5) 9 }) 10 11 endpoint_df = pd.DataFrame(endpoint_data) 12 endpoint_df.to_csv("endpoints.csv", index=False) 13 print("-----") 14 print(" Dummy Endpoint datasets has been created successfully with 50 15 records.") 16 print("-----") 17 18 ✓ [41] < 10 ms 19 20 ----- 21 Dummy Endpoint datasets has been created successfully with 50 records. 22 ----- </pre>
3	Data collection advise for data backups monitoring.	Step3: • This given code in screenshot creates backup dataset for backup monitoring and recording the backup coverages. • It consists of encryption status, last backups timestamps, and validation results. • It helps to maintain integrity, data availability, and recovery readiness. Creating Backup Dataset <pre> 1 backup_data = [] 2 3 for i in range(50): 4 backup_data.append({ 5 "system": f"System-{i+1:02}", 6 "backup_coverage": random.randint(70, 100), 7 "encryption_status": random.choice(["Yes", "No"]), 8 "last_backup": (base_time - timedelta(hours=random.randint(1, 9 48))).strftime("%Y-%m-%d %H:%M"), 10 "validation_status": random.choice(["Pass", "Fail"]) 11 }) 12 13 backup_df = pd.DataFrame(backup_data) 14 backup_df.to_csv("backups.csv", index=False) 15 print("-----") 16 print(" Dummy Backup datasets has been created successfully with 50 17 records.") 18 print("-----") 19 20 ✓ [42] < 10 ms 21 22 ----- 23 Dummy Backup datasets has been created successfully with 50 records. 24 ----- </pre>

4	Data collection advise for staff readiness monitoring.	Step4: - Staff readiness dataset is created by using the code given below. - This dataset focuses on the human aspect of cyber security. - It records the user roles, training completion status, phishing awareness scores, and other risk levels. - This data helps in monitoring and identifying high-risk user and supports targeted security awareness in training programs.  <pre> 1 staff_data = [] 2 3 for i in range(50): 4 staff_data.append({ 5 "staff_id": f"S{i+1:03}", 6 "role": random.choice(roles), 7 "training_status": random.choice(["Completed", "Pending"]), 8 "phishing_score": random.randint(50, 100), 9 "risk_level": random.choice(risk_levels) 10 }) 11 12 staff_df = pd.DataFrame(staff_data) 13 staff_df.to_csv("staff_readiness.csv", index=False) 14 print("-----") 15 print(" Dummy Staff datasets has been created successfully with 50 records.") 16 print("-----") 17 ✓ [45] < 10 ms 18 19 ----- 20 Dummy Staff datasets has been created successfully with 50 records. 21 ----- </pre>
5	Prototype dashboard using two input files.	Step5: - A prototype cybersecurity dashboard is created using two .csv files that we have created in above task. - The files used for creating Dashboard are Network_traffic.csv and endpoints.csv - These datasets are loaded using pandas' libraries.  <pre> 13 14 # LOADING DATA OF NETWORK_TRAFFIC.CSV AND ENDPOINTS.CSV 15 network = pd.read_csv("network_traffic.csv") 16 endpoints = pd.read_csv("endpoints.csv") 17 </pre>

		• And stramlit library and stramlit application loads these datasets and visualizes the key metrics, enabling the real-time monitoring of network activity. • It loads the data and visualizes the dashboard inside the website. Local Government Cyber Security Dashboard Prototype dashboard using two input files (network & endpoints) Security Overview <table><tr><td>Total Network Events</td><td>Endpoints with Malware Alerts</td><td>Firewalls Enabled</td></tr><tr><td>50</td><td>41</td><td>29</td></tr></table> • The dashboard shows the local government Cybersecurity dashboard. It is a prototype dashboard showing the clear visualization of Network_traffic.csv and endpoint.csv. • Inside the security overview part, it shows total network events is 50 endpoints malware alerts is 41 and enabled firewalls is 29. The dashboard integrates both the data sets: • It shows the final and overall posture of the security • Monitors the behaviours of the network traffic. • Monitors the end points of the security health.	Total Network Events	Endpoints with Malware Alerts	Firewalls Enabled	50	41	29
Total Network Events	Endpoints with Malware Alerts	Firewalls Enabled						
50	41	29						
6	Effective layout and design of the dashboard.	The dashboard uses STREAMLIT's layout system in an effective way, this allows the dashboard to visualize in full screen. <pre># defining the rows and columns for the visualization st.header("Security Overview") c1, c2, c3 = st.columns(3) c1.metric("Total Network Events", len(network)) c2.metric("Endpoints with Malware Alerts", endpoints[endpoints["malware_alerts"] > 0].shape[0]) c3.metric("Firewalls Enabled", endpoints[endpoints["firewall_status"] == "On"].shape[0])</pre>						

		<pre># Creating clear Visualization of Network monitoring st.header(" Network Traffic Monitoring") col1, col2 = st.columns(2)</pre> <pre># Dashboard presenting ENDPOINT MONITORING st.header(" Endpoint Security Monitoring") col3, col4 = st.columns(2)</pre> <pre>c1.metric("Total Network Events", len(network)) c2.metric("Endpoints with Malware Alerts", endpoints[endpoints["malware_alerts"] > 0].shape[0]) c3.metric("Firewalls Enabled", endpoints[endpoints["firewall_status"] == "On"].shape[0])</pre> This is the columns layout matrices which keeps the chart side by side and improves the readability and shows professional type of appearance. And it also helps for avoiding long horizontal and vertical scrolling. Network Traffic Monitoring Network Protocol Distribution <table><tr><th>Protocol</th><th>Percentage</th></tr><tr><td>UDP</td><td>38%</td></tr><tr><td>ICMP</td><td>28%</td></tr><tr><td>TCP</td><td>34%</td></tr></table> Network Throughput Over Time <table><tr><th>Time</th><th>Throughput</th></tr><tr><td>0</td><td>150</td></tr><tr><td>10</td><td>200</td></tr><tr><td>20</td><td>180</td></tr><tr><td>30</td><td>220</td></tr><tr><td>40</td><td>160</td></tr><tr><td>50</td><td>210</td></tr><tr><td>60</td><td>190</td></tr><tr><td>70</td><td>230</td></tr><tr><td>80</td><td>170</td></tr><tr><td>90</td><td>200</td></tr><tr><td>100</td><td>180</td></tr></table> • The pie chart shows the network protocol of the network traffic. • UDP protocol is 38%, ICMP protocol is 28%, and TCP protocol is 34% Endpoint Security Monitoring Malware Alerts per Endpoint <table><tr><th>Endpoint</th><th>Alerts</th></tr><tr><td>1</td><td>2</td></tr><tr><td>2</td><td>4</td></tr><tr><td>3</td><td>3</td></tr><tr><td>4</td><td>2</td></tr><tr><td>5</td><td>3</td></tr><tr><td>6</td><td>4</td></tr><tr><td>7</td><td>2</td></tr><tr><td>8</td><td>3</td></tr><tr><td>9</td><td>4</td></tr><tr><td>10</td><td>2</td></tr></table> Firewall Status Distribution <table><tr><th>Status</th><th>Count</th></tr><tr><td>On</td><td>20</td></tr><tr><td>Off</td><td>10</td></tr></table> This is a Prototype dashboard of Network traffic and Endpoint security monitoring. The bar chart shows the visualization of malware inside the first graph and firewall status of the device in second chart.	Protocol	Percentage	UDP	38%	ICMP	28%	TCP	34%	Time	Throughput	0	150	10	200	20	180	30	220	40	160	50	210	60	190	70	230	80	170	90	200	100	180	Endpoint	Alerts	1	2	2	4	3	3	4	2	5	3	6	4	7	2	8	3	9	4	10	2	Status	Count	On	20	Off	10	Total
Protocol	Percentage																																																														
UDP	38%																																																														
ICMP	28%																																																														
TCP	34%																																																														
Time	Throughput																																																														
0	150																																																														
10	200																																																														
20	180																																																														
30	220																																																														
40	160																																																														
50	210																																																														
60	190																																																														
70	230																																																														
80	170																																																														
90	200																																																														
100	180																																																														
Endpoint	Alerts																																																														
1	2																																																														
2	4																																																														
3	3																																																														
4	2																																																														
5	3																																																														
6	4																																																														
7	2																																																														
8	3																																																														
9	4																																																														
10	2																																																														
Status	Count																																																														
On	20																																																														
Off	10																																																														

References:

1. <https://www.geeksforgeeks.org/data-science/data-science-101-an-easy-introduction/>
2. https://books.apple.com/nz/book/data-science-for-cyber-security/id1440282260?utm_source=chatgpt.com